

Intern Feedback Form

A simple web form where an intern can submit weekly feedback. The form validates input on the client side, sends it to a mock REST API with a `POST` request, and clearly shows loading, success, and error states. It also fetches and displays the 5 most recent posts from the API.

Features

- Accessible HTML form with labeled fields (name, email, category, rating, message)
- Client-side validation with inline error messages (no `alert()` popups)
- `POST` request to a mock API using `fetch()`
- Loading state (disabled button + spinner) while the request is in flight
- Success state showing the returned ID (e.g. "Thanks, Sara! Saved with ID 101.")
- Error state that keeps the user's input so they can retry
- Bonus: fetches and lists the latest 5 posts below the form
- Bonus: responsive layout, tested at 375px and 1280px widths

How to run it

This project now uses **json-server** to actually save and persist feedback locally (instead of a fake mock API), so you need to run a small local server alongside the page.

1. Install json-server (one-time, needs Node.js installed):
2. `npm install -g json-server`
3. From the project folder, start the server:
4. `json-server --watch db.json --port 3000`

Keep this terminal window open — it's your local API, running at `http://localhost:3000`.

5. Open `index.html` in your browser (just double-click it, or use a Live Server extension).
6. Fill in the form and submit — your feedback is now really saved into `db.json` and will still be there even after you refresh the page.

Which API I used, and why

I switched from JSONPlaceholder to **json-server** because JSONPlaceholder is a fake API that never actually stores anything — every `GET` request returns the same fixed sample data no matter what you `POST`. `json-server` runs a real local REST API backed by a simple `db.json` file, so submissions are genuinely saved and show up in the "Latest Feedback" list even after a page refresh, which felt more true to how a real feedback form would behave.

Screenshots

Validation error:

Week 2 Program

Intern Feedback Form

Tell us how your week went — it takes less than a minute.

Full name

Name must be 3–50 characters.

Email

Enter a valid email address.

Category

-- Select --

Please choose a category.

Rating (1–5)

-- Select --

Please choose a rating.

Message

what went well, what was hard...

0 / 500

Message must be 10–500 characters.

Submit

Latest Feedback

#3Znlb75uWF0 — Good but need improvement

Successful submission:

Week 2 Program

Intern Feedback Form

Tell us how your week went — it takes less than a minute.

Full name

e.g. Sara Ahmed

Email

sara@example.com

Category

-- Select --

Rating (1–5)

-- Select --

Message

what went well, what was hard...

0 / 500

Submit

Thanks, Kahn G! Saved with ID 3Znlb75uWF0.

Latest Feedback

#3Znlb75uWF0 — Good but need improvement

What I learned / what was hard

Building this form gave me an opportunity by connecting a frontend to a real API instead of just working with static data, so the biggest thing I learned was how the `fetch()` request/response cycle actually works — sending JSON with the right `Content-Type` header, and `await`-ing the response before I can use it. The part that tripped me up at first was realizing `fetch()` does not throw an error on a 404 or 500 response, only on network failures — I had to explicitly check `response.ok` myself and throw my own error to get the failure state to trigger. Managing the three UI states (loading, success, error) was also trickier than I expected, since I had to remember

to disable the button before the request and re-enable it in a `finally` block so it doesn't stay stuck if something goes wrong. Writing the validation in JavaScript instead of relying on HTML's built-in `required` attributes took more code, but it let me control exactly where and how error messages appear next to each field. Overall this project made REST APIs feel much less abstract than they did after just reading about them.